

Санкт-Петербургский государственный университет

Кисельков Денис Андреевич

Выпускная квалификационная работа

Разработка образовательного комплекса
виртуального устройства и минимальной
ОС на архитектуре RISC-V

Уровень образования: магистратура

Направление *09.04.04 «Программная инженерия»*

Основная образовательная программа *ВМ.5666.2022 «Программная инженерия»*

Научный руководитель:
доцент кафедры системного программирования, к. ф.-м. н., Луцив Д. В.

Рецензент:
Ведущий инженер-программист, ООО «Специальный Технологический Центр»,
Жиданов К. А.

Санкт-Петербург
2026

Saint Petersburg State University

Denis Kiselkov

Master's Thesis

Development of an educational complex of a
virtual device and a minimal OS on the
RISC-V architecture

Education level: master

Speciality *09.04.04 "Software Engineering"*

Programme *BM.5666.2022 "Software Engineering"*

Scientific supervisor:
Associate Professor of the Department of Systems Programming, Candidate of Physical and
Mathematical Sciences, D.V. Lutsiv

Reviewer:
Leading Software Engineer, Special Technology Center LLC, Zhidanov K. A.

Saint Petersburg
2026

Оглавление

Введение	5
1. Постановка задачи	8
2. Анализ предметной области	10
2.1. Открытые архитектуры центральных процессоров	10
2.2. Легковесные UNIX-подобные операционные системы . .	11
2.3. Компактные компиляторы языка Си	13
2.4. Учебные компьютерные платформы: от Nand2Tetris до FPGA-курсов	14
2.5. Выводы и требования к учебному стенду	17
3. Архитектура программного комплекса	19
3.1. Назначение комплекса	19
3.2. Компонентная структура	20
3.3. Сценарий работы	22
3.4. Технологический стек и платформа	23
3.5. Базовые архитектурные решения	23
4. Реализация компонентов комплекса	26
4.1. Разработка синтезируемого ядра RISC-V RV32I на SystemVerilog	26
4.2. Моделируемая вычислительная платформа: память, пе- риферия, загрузчик	32
4.3. Адаптация и сборка операционной системы xv6	35
4.4. Адаптация и интеграция компилятора TinyCC	39
4.5. Инфраструктура воспроизводимой сборки и отладки . .	40
5. Тестирование, верификация и методическое обеспечение	45
5.1. Методика тестирования и конфигурация стенда	45
5.2. Результаты функционального тестирования и верификации	47
5.3. Учебно-методические материалы	48

6. Заключение	50
Список литературы	52

Введение

Политика импортозамещения, подкрепленная в 2025–2026 годах субсидированием разработки программного обеспечения [17], увеличивает потребность российской экономики в отечественных аппаратно-программных решениях. Санкционные ограничения и прекращение официальной поддержки предыдущих версий Windows, стимулирующее переход на Windows 11, способствуют росту доли операционной системы Linux на рынке [23], тогда как доля программного обеспечения в отрасли информационных технологий достигла 40% в 2024 году при одновременном сокращении аппаратной составляющей. Эти изменения повышают спрос на разработчиков, владеющих фундаментальными принципами построения компьютерных систем — процессов, памяти, операционных систем — и способных реализовывать их на практике.

Одновременно активно развиваются технологические направления, требующие глубоких системных знаний: тяжёлые мультироторные дроны, робототехника, интернет вещей, процессоры для вычислительной техники, где критически важны понимание архитектуры набора инструкций (ISA), устройства центрального процессора (арифметико-логического устройства, регистров), взаимодействия с периферией и основ функционирования операционных систем. Рынок труда фиксирует значительный дефицит таких специалистов: в сфере информационных технологий открыто до миллиона вакансий, в микроэлектронике не хватает 5,3 тыс. инженеров (по данным 2024 г.), причем более 60% позиций остаются незакрытыми из-за пробелов в системном мышлении соискателей [22].

В российских университетах (БГТУ, СПбГУ и других) теория компьютерных систем в рамках дисциплин «Операционные системы» и «Архитектура ЭВМ» излагается подробно, однако материал подаётся фрагментарно. Курсы по архитектуре ЭВМ рассматривают модель фон Неймана, логические элементы, память и базовые компоненты процессоров; дисциплины по операционным системам — планировщики, прерывания и системные вызовы. В учебном процессе отсутствует нагляд-

ная демонстрация взаимосвязей между уровнями организации вычислительной системы: каким образом логические вентили образуют центральный процессор, как цикл операционной системы реализуется на уровне машинного кода, каким образом системные абстракции проявляются в ядре. Студенты приобретают практический опыт в симуляторах логических схем (Logisim, Digital), однако не переходят к более сложным и содержательным примерам — отладке программ, изучению минимальных операционных систем или виртуальных устройств. В результате выпускники владеют отдельными абстрактными концепциями, но затрудняются применять их для решения прикладных задач, что ограничивает их участие в проектной деятельности.

Открытые образовательные проекты — nandgame, nand2tetris, учебная ОС xv6, ранние версии Linux и GNU Mes — демонстрируют эффективный системный подход и представляют качественные практические примеры. Вместе с тем эти проекты недостаточно встроены в учебные программы, а начинающим разработчикам трудно самостоятельно построить на их основе целостный курс изучения материала.

Настоящая магистерская работа направлена на устранение указанного разрыва. Предлагается разработать учебный программный комплекс, объединяющий три ключевых компонента в единой воспроизводимой среде:

- Виртуальный RISC-V процессор на SystemVerilog с тактовой трассировкой и выводом временных диаграмм;
- Минимальную UNIX-подобную операционную систему xv6, выполняющуюся непосредственно на виртуальном процессоре;
- Легковесный компилятор TinyCC, портированный для работы внутри xv6, позволяющий компилировать и выполнять пользовательские C-программы.

Комплекс реализует учебный процесс: обучающийся собирает операционную систему xv6 с помощью RISC-V инструментария, загружает её

в виртуальный процессор, затем внутри гостевой ОС запускает встроенный компилятор TinyCC, пишет программу на языке Си, компилирует её прямо в среде xv6 и наблюдает потактовое выполнение — стадия выполнения инструкции, обращения к памяти, обработку прерываний и системных вызовов — вплоть до вывода результата на виртуальный терминал. Тем самым в рамках одного лабораторного стенда связываются курсы «Архитектура ЭВМ», «Системное программное обеспечение» и «Операционные системы». Для подтверждения работоспособности комплекса планируется провести его тестирование на студенческих группах, а также подготовить комплект учебно-методических материалов и рекомендации по использованию смежных проектов. Программные компоненты комплекса реализуются на языках Verilog и Си и документируются с целью подтверждения возможности создания предложенных виртуальных устройств и их изучения в образовательном процессе.

1. Постановка задачи

Цель выпускной квалификационной работы — разработка учебного программного комплекса, объединяющего виртуальный RISC-V процессор, минимальную UNIX-подобную операционную систему и легковесный компилятор для языка C. Комплекс предназначен для наглядной демонстрации жизненного цикла разработки и функционирования программ - от компиляции исходного кода до выполнения на процессоре. Комплекс предполагается использовать в рамках университетских курсов по архитектуре ЭВМ и системному программированию.

Для осуществления этой цели были сформулированы следующие задачи.

1. Провести обзор открытых процессорных архитектур , легковесных UNIX-подобных ОС и компактных C-компиляторов с точки зрения применимости в образовательных проектах; на основе обзора выработать требования к учебному стенду и обосновать выбор технологического стека.
2. Спроектировать архитектуру программного комплекса, определяя взаимодействие виртуального процессора с периферией, ОС и легковесным C-компилятором спроектировать средства наблюдения и отладки.
3. Реализовать на SystemVerilog синтезируемое ядро RISC-V RV32I с полной поддержкой целочисленной арифметики и рядом других возможностей.
4. Выполнить адаптацию и сборку исходного кода UNIX-подобной ОС для созданного процессорного ядра, обеспечив корректную загрузку и выполнение ядра на виртуальном процессоре.
5. Адаптировать легковесный компилятор языка C для работы внутри гостевой ОС, обеспечив возможность компиляции и выполнения пользовательских C-программ на моделируемом RISC-V процессоре.

6. Провести тестирование и верификацию всех компонентов комплекса на задачах управления процессами, памятью и вводом-выводом.
7. Подготовить учебно-методические материалы для использования комплекса в учебном процессе.

2. Анализ предметной области

2.1. Открытые архитектуры центральных процессоров

Выбор процессорной архитектуры для учебного стенда, призванного демонстрировать полный цикл разработки и выполнения программ, определяется не только функциональной полнотой, но и степенью доступности спецификаций, инструментария и образовательных ресурсов. Предпочтение отдаётся архитектурам, не обременённым лицензионными ограничениями и допускающим свободное воспроизведение как в симуляции, так и в кремнии. Среди свободно распространяемых ISA выделяются RISC-V, исторически открытые версии MIPS, OpenRISC и SPARC V8.

Архитектура MIPS на протяжении десятилетий служила академическим целям и до сих пор встречается в учебных курсах по организации ЭВМ [13]. Однако развитие её открытых вариантов фактически остановилось, а современные реализации требуют приобретения коммерческих лицензий, что противоречит принципу воспроизводимости. Семейство ARM, доминирующее во встраиваемых системах, остаётся проприетарным, и его свободное использование ограничено лицензионными соглашениями [1].

Архитектура RISC-V лишена названных ограничений [11]. Её спецификации распространяются под свободными лицензиями, разрешающими любые реализации без отчислений. Модульная структура ISA позволяет ограничиться базовым подмножеством целочисленных инструкций RV32I, которое сочетает компактность, достаточную для понимания студентами, с полноценной поддержкой современных RISC-принципов: фиксированная длина инструкции, регистровая архитектура, простые способы адресации. Зрелая экосистема (GCC, LLVM, эмуляторы, отладчики) и активное сообщество делают RISC-V обоснованным выбором для создания открытого учебного процессора.

2.2. Легковесные UNIX-подобные операционные системы

Операционная система (ОС) представляет собой комплекс программного обеспечения, управляющий аппаратными ресурсами вычислительной установки и предоставляющий интерфейс взаимодействия между пользователем и устройством. К её фундаментальным функциям относятся: загрузка и исполнение прикладных программ с образованием процессов; распределение процессорного времени между вычислительными задачами (планирование); управление оперативной памятью, включая механизмы виртуальной памяти, позволяющие расширять адресное пространство за счёт дисковой подсистемы; организация файловой системы и регламентация доступа к данным на постоянных носителях; взаимодействие с периферийным оборудованием через драйверы; предоставление пользовательского интерфейса — командной строки или графической среды. В учебном стенде, нацеленном на демонстрацию цикла работы программ, операционная система должна не только реализовывать перечисленные функции, но и обладать обозримым, хорошо документированным исходным кодом, допускающим детальный анализ в рамках семестрового курса.

Традиционные UNIX-подобные системы общего назначения, такие как GNU/Linux, не удовлетворяют последнему требованию. Современное ядро Linux содержит миллионы строк кода, а его изучение затруднено даже на уровне ранних версий. Первая версия ядра (0.01), выпущенная Линусом Торвальдсом в сентябре 1991 г. [16], являлась прототипом с базовой поддержкой управления памятью, многозадачности, элементарной файловой системы и взаимодействия через терминал. Однако её документация оставалась фрагментарной, а сообщество разработчиков в тот период было ориентировано скорее на эксперименты с кодом, чем на его систематическое описание. Компактные сборки Linux с использованием BusyBox [2] не решают проблему: BusyBox объединяет множество стандартных утилит в одном исполняемом файле, скрадывая границы между отдельными командами и затрудняя понимание их

взаимодействия.

Ряд исторических и специализированных систем также не пригоден для образовательных целей. Операционная система CP/M [3], созданная в 1973 г. на языке PL/M для процессоров Intel 8080 и Zilog Z80, стала фактическим стандартом 8-битных микрокомпьютеров, однако её архитектура безнадёжно устарела, а оригинальные материалы труднодоступны. Отечественная MicroDOS [8] для персональных компьютеров БК-0010 не поддерживается с 1992 г., имеет закрытый исходный код и лишена возможности модификации. Embox [4] представляет собой ОС реального времени, нацеленную на встроенные системы с ограниченными ресурсами. Глубокая привязка к конкретным аппаратным платформам и специфика задач реального времени не позволяют на её основе изучать универсальные механизмы управления процессами и файловыми подсистемами. Проект GNU Mes [6] решает задачу самодостаточной сборки компиляторов (бутстрэппинга) и предоставляет интерпретатор Scheme вместе с простым компилятором C, однако не содержит многих компонентов полноценной ОС — подсистем управления процессами, сетевого стека и т. п.

Шестая редакция операционной системы Unix (Unix V6), выпущенная Bell Labs в 1975 г. для архитектуры PDP-11 [18], на протяжении десятилетий служила учебным пособием. Благодаря небольшому объёму (порядка 10 000 строк) и ясной структуре она подходила для изучения принципов проектирования операционных систем. Вместе с тем код написан на раннем диалекте языка C и ориентирован на устаревшее оборудование; отсутствуют поддержка многопоточности и развитые механизмы управления памятью, что ограничивает перенос получаемых знаний на современные платформы.

Указанные противоречия разрешает учебная операционная система Hxv6, разработанная в Массачусетском технологическом институте [14]. Hxv6 воспроизводит архитектуру Unix V6, однако полностью переписана на ANSI C, снабжена подробными комментариями, поддерживает многопроцессорность и адаптирована для архитектур x86 и RISC-V. Объём исходного кода составляет около 9000 строк — величина, допускаю-

щая детальное изучение в течение нескольких академических занятий. Система реализует защищённое ядро, виртуальную память, файловую систему и стандартный интерфейс системных вызовов, а благодаря запуску в эмуляторах (например, QEMU) становится удобным объектом трассировки и отладки. Некоторым ограничением можно считать отсутствие поддержки исполнения на физическом оборудовании, однако для учебного стенда, ориентированного на Verilog-модель процессора, эмуляционный характер исполнения является скорее преимуществом.

Альтернативные учебные ОС, такие как Minix с микроядерной архитектурой [9], или uClinux, нацеленная на системы без аппаратной поддержки виртуальной памяти, либо добавляют избыточную концептуальную сложность, либо не охватывают весь спектр механизмов, характерных для полноценного UNIX-ядра. Таким образом, Xv6, сочетающая простоту, полноту и прямую поддержку RISC-V, выступает более подходящей операционной системой для проектируемого стенда.

2.3. Компактные компиляторы языка Си

Включение в стенд компилятора, обеспечивающего трансляцию прикладных программ в код целевой архитектуры, необходимо для реализации сценария «исходный код — исполняемый образ — симуляция». Компилятор должен, с одной стороны, поддерживать стандарт языка Си, с другой — обладать умеренным объёмом исходного кода, допускающим адаптацию бэкенда. Промышленные пакеты GCC и LLVM этому требованию не удовлетворяют: их внутреннее устройство чрезвычайно сложно, а модификация генератора кода для новой, пусть и похожей, архитектуры трудоёмка.

Среди легковесных компиляторов выделяются два кандидата: LCC (Little C Compiler) и TinyCC. LCC в течение многих лет применялся в образовании благодаря ясной архитектуре и чёткому разделению стадий компиляции, однако развитие проекта остановилось [7], и генератор кода для RISC-V в нём отсутствует. TinyCC (Tiny C Compiler) [12], напротив, продолжает поддерживаться, реализован на языке Си, имеет

сравнительно небольшой размер (около 200 строк) и изначально проектировался с прицелом на высокую скорость компиляции и возможность встраивания. Гибкая система описания целевых платформ позволила относительно легко добавить поддержку архитектуры RISC-V, что и было выполнено в рамках данной работы. Совокупность этих свойств делает TinyCC соответствующим критериям простоты и расширяемости.

2.4. Учебные компьютерные платформы: от Nand2Tetris до FPGA-курсов

Учебные компьютеры представляют собой виртуальные или синтезируемые модели вычислительных устройств, предназначенные для освоения принципов архитектуры ЭВМ без необходимости приобретения физического оборудования. При анализе таких платформ целесообразно учитывать несколько ключевых характеристик.

Прежде всего, важна организация набора инструкций (ISA): принадлежность к классу RISC, CISC или оригинальной архитектуре; разрядность (8, 16 или 32 бита); набор поддерживаемых операций — арифметических, логических, операций с памятью, переходов; доступные режимы адресации и наличие системного стека. Эти свойства непосредственно определяют сложность модели и степень её приближения к реальным процессорам.

Состав компонентов также влияет на полноту отражения аппаратных механизмов. Типичный учебный компьютер включает программный счётчик, регистр состояния с флагами, стек, арифметико-логическое устройство (ALU) и блок управления. Более развитые модели могут содержать многоступенчатый конвейер, кэш-память, средства моделирования прерываний, таймеров, контроллеров прямого доступа к памяти (DMA) и даже элементы многоядерности. Совокупность этих компонентов определяет, насколько детально воспроизводятся конвейеризация, иерархия памяти и реакция на внешние события.

Способ реализации заметно влияет на образовательный эффект.

Платформа может быть описана на языках аппаратной логики (Verilog, VHDL, SystemVerilog) и предназначаться для синтеза на ПЛИС, что даёт потенциальную возможность физического воплощения. Альтернативой служит программный эмулятор на языках высокого уровня (Python, C, Java), представляющий собой чисто абстрактную модель без прямой связи с аппаратурой. Первый подход обеспечивает наглядность «железной» стороны архитектуры, второй — быстроту запуска и простоту модификации.

Наконец, интерфейс и интерактивность определяют удобство работы. Наличие текстовой консоли (ассемблерная консоль, командная строка) и/или графического интерфейса с визуализацией регистров, памяти, шин, поддержкой пошагового выполнения и отладки существенно усиливает обучающий эффект, делая поведение модели управляемым и понятным.

Существующие учебные платформы можно расположить на спектре от простейших абстракций до синтезируемых процессорных ядер, способных исполнять полноценную операционную систему.

Модель «Маленького человечка» (Little Man Computer, LMC), предложенная Стюартом Мэдником в 1965 г. [19], представляет аналогию почтового отделения: в закрытой комнате «человечек» выполняет инструкции, манипулируя данными в ячейках памяти. Память содержит 100 ячеек (адреса 0–99), каждая из которых хранит трёхзначные десятичные числа. Ввод и вывод осуществляются через воображаемые лотки INBOX и OUTBOX, арифметические операции ограничены сложением и вычитанием. LMC отлично подходит для начальной демонстрации принципа хранимой программы, однако не даёт представления о конвейере, иерархии памяти или взаимодействии с периферией.

Архитектура SIC/XE (Simplified Instructional Computer Extra Equipment) [15] расширяет базовую модель SIC и применяется в ряде учебников по системному программированию. Объём памяти увеличен до 1 Мбайт, организованной 8-битными байтами; машинное слово состоит из трёх последовательных байтов (24 бита). Девять регистров, включая базовый и регистры с плавающей запятой, а также четыре формата

инструкций и несколько режимов адресации (прямая, индексированная, базовая) приближают модель к реальным процессорам. Набор инструкций дополнен операциями с плавающей запятой и системными вызовами для обработки прерываний. Однако реализация существует лишь в форме программного симулятора, что не позволяет продемонстрировать связь с физической аппаратурой.

Модель «E14» [20] ориентирована на изучение многоядерной архитектуры и параллельного программирования. В ней реализована несимметричная конфигурация, где один из процессоров выполняет функции ведущего. Система команд совместима с предшествующей моделью «E97», что упрощает переход от однопроцессорных концепций к многопроцессорным. Межпроцессорный обмен использует порты, общую шину и механизмы прямого доступа к памяти. «E14» работает под управлением Windows как прикладная программа и не рассчитана на запуск операционной системы, что ограничивает её применение при изучении системного ПО.

Отечественная учебная модель ЭВМ [21] делает акцент на работе кэш-памяти. В её состав входят процессор, оперативная память и сверхоперативная память, объединяющая регистры общего назначения и кэш, а также устройства ввода/вывода. Модель наглядно демонстрирует ускорение доступа к данным, но не затрагивает верхние уровни организации вычислительного процесса.

Проект Nand2Tetris [5] реализует комплексный подход, проводя обучающегося через всю иерархию вычислительной системы: от логического вентиля NAND до операционной системы и компилятора. Аппаратная часть базируется на 16-битной архитектуре Nask с 17 инструкциями и поддержкой экранного вывода и клавиатуры. Программная часть включает ассемблер, виртуальную машину со стековой организацией и компилятор объектно-ориентированного языка Jack. Несмотря на глубину проработки, полное прохождение курса требует более недели практических занятий, что трудно вписать в типовой семестровый график. Кроме того, использование собственной архитектуры и языков программирования изолирует получаемые навыки от промышленных

стандартов. Интерактивная игра Nand Game, решающая сходную задачу в упрощённой форме (построение вентилей, ALU, регистров, памяти на D-триггерах и процессора начиная с NAND-вентиля), даёт лишь поверхностное представление и недостаточна для университетского курса.

На другом полюсе находятся синтезируемые ядра промышленного уровня. Ядро RISC-V CPU Core (RV32IM) UltraEmbedded представляет собой 32-битный процессор на Verilog, поддерживающий набор инструкций RV32IM, пользовательский, супервизорный и машинный режимы, систему управления памятью (MMU), эмуляцию атомарных операций и интерфейсы AXI [10]. Оно позволяет запускать ОС Linux и проверено с помощью средств co-simulation и Google RISC-V-DV. Однако полнота и сложность этого ядра делают его скорее объектом изучения готовой архитектуры, нежели платформой для поэтапного освоения принципов проектирования.

Таким образом, ни одна из рассмотренных учебных платформ не объединяет открытый процессор со стандартной RISC-архитектурой, легковесную UNIX-подобную ОС и компактный компилятор, обеспечивая при этом демонстрацию цикла разработки и выполнения программ. Этот пробел обосновывает необходимость создания нового программного комплекса.

2.5. Выводы и требования к учебному стенду

Обзор открытых процессорных архитектур показал, что RISC-V RV32I является оптимальной основой для учебного процессора благодаря свободной лицензии, модульности и развитой экосистеме. Среди легковесных UNIX-подобных ОС выделяется Xv6: она переписана на ANSI C, хорошо документирована, содержит около 9000 строк кода и поддерживает архитектуру RISC-V, что делает её подходящим объектом для изучения механизмов управления процессами, памятью и вводом-выводом. В качестве компилятора языка Си обоснован выбор TinyCC, сочетающего умеренный объём исходного кода с гибкостью адаптации бэкенда. Анализ учебных компьютерных платформ вы-

явил отсутствие решений, интегрирующих все три компонента в единый стенд.

На основании проведённого анализа формулируются следующие требования к проектируемому комплексу:

1. Процессорное ядро должно реализовывать спецификацию RISC-V RV32I (базовый целочисленный набор инструкций), быть синтезируемым на ПЛИС и обеспечивать генерацию сигналов трассировки.
2. Операционная система — Xv6, собранная для целевой архитектуры и способная выполняться на виртуальном процессоре написанном на verilog.
3. Компилятор — TinyCC интегрированный в сборочную цепочку.
4. Комплекс обязан реализовывать маршруты: компиляция программы и ее запуск на виртуальном процессоре; запуск xv6 на виртуальном процессоре и компиляция программы внутри с помощью TinyCC.
5. Для наглядности необходимо предоставить средства трассировки (VCD-файлы) и отладки состояния процессора, памяти и периферийных устройств.
6. Воспроизводимость платформы должна обеспечиваться унифицированным сборочным окружением (Docker-образ) и едиными репозиториями всех компонентов.
7. Комплекс сопровождается учебно-методическими материалами, описывающими архитектуру, процедуру сборки и сценарии лабораторных работ.

Детальная архитектура стенда, отвечающего перечисленным требованиям, рассматривается в следующем разделе.

3. Архитектура программного комплекса

3.1. Назначение комплекса

Учебный программный комплекс предназначен для демонстрации жизненного цикла разработки и выполнения программ в рамках лабораторных и лекционных занятий по архитектуре ЭВМ и операционным системам. Комплекс позволяет проследить преобразование исходного кода на языке Си в машинные инструкции, загрузку исполняемого образа в память виртуального процессора и последующее исполнение под управлением операционной системы с выводом на виртуальную периферию.

Комплекс решает следующие образовательные задачи:

- объединение в едином стенде трёх дисциплин: архитектуры процессора (RISC-V), системного программирования (легковесный компилятор TinyCC) и операционных систем (адаптированное ядро xv6);
- обеспечение потактовой фиксации состояния процессора в VCD-файл с возможностью визуализации в GTKWave;
- воспроизводимость экспериментов благодаря контейнеризации и хранению артефактов в системе контроля версий;
- автоматическая верификация консольного вывода путём сравнения с эталоном, полученным на QEMU;
- поддержка двух режимов выполнения: `run_prog` — запуск «голых» Си-программ без операционной системы, `run_xv6` — запуск многозадачной ОС xv6.

Таким образом, комплекс предоставляет единую среду, в которой студент может проследить путь от исходного текста программы до деталей её исполнения на вычислительной системе.

3.2. Компонентная структура

Архитектура комплекса организована в виде четырёх взаимодействующих уровней: инструментального, моделируемого, операционного и наблюдательного. Взаимосвязи между компонентами в динамике их взаимодействия отражены на диаграмме рисунок 1.

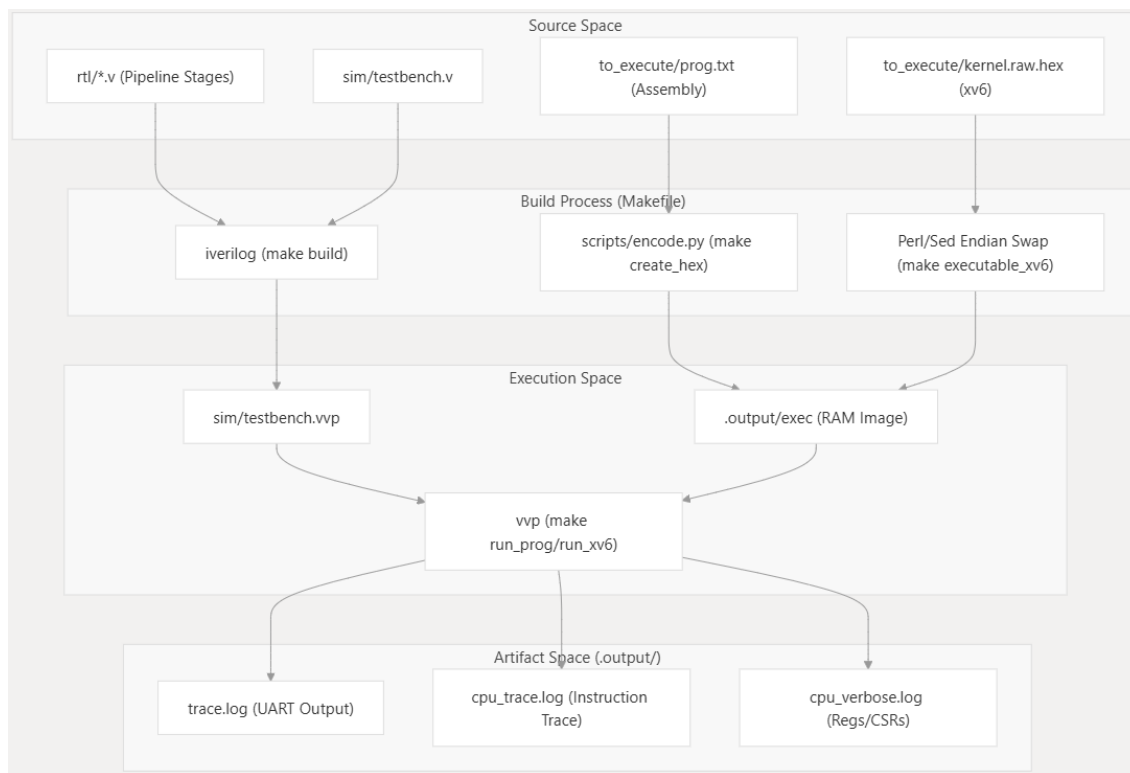


Рис. 1: Диаграмма уровней

Инструментальный уровень включает компилятор `riscv32-unknown-elf-gcc` для сборки ядра и пользовательских программ, утилиты преобразования ELF-файлов в hex-формат, сценарий консольного ввода `console_script.txt`, а также сборочное окружение (Makefile, Docker). Результатом работы уровня является образ вычислительной системы — бинарный код и данные, подготовленные для загрузки в виртуальный процессор.

Моделируемый уровень представляет собой виртуальную платформу, описанную на SystemVerilog. Модуль верхнего уровня `top.sv` инстанцирует процессор, память, периферийные устройства и загрузчик. Процессор реализует многотактовое ядро RV32I с токеной синхронизацией; в каждый момент времени активна только одна стадия

конвейера, управление передаётся сигналом-токеном по кольцу PCU → IF → ID → EX → MEM → WB → PCU. Модуль `memory_unit` задаёт единое адресное пространство со следующей фиксированной картой:

- 0x80000000 -- 0x88000000 — оперативная память (2 Мбайт);
- 0x10000000 -- 0x1000000F — регистры UART;
- 0x02000000 -- 0x0200FFFF — регистры таймера CLINT (`mtime`, `mtimecmp`).

Дешифратор адресов маршрутизирует обращения к оперативной памяти или периферийным устройствам. Модуль `uart_console_sim` эмулирует приёмопередатчик: при чтении регистра данных возвращает очередной байт из сценария ввода, при записи сохраняет байт в выходной лог и сверяет его с эталоном. Модуль `clint` содержит счётчик `mtime` и регистр сравнения `mtimecmp`; при достижении равенства генерируется сигнал прерывания `timer_irq`. Загрузчик, отрабатывающий на нулевом такте, заполняет оперативную память содержимым hex-файла посредством системной функции `$readmemh`.

Операционный уровень объединяет программные компоненты, исполняемые на виртуальном процессоре. В конфигурации `run_prog` это скомпилированная программа без операционной системы, напрямую обращающаяся к UART через отображённые в память регистры ввода-вывода (MMIO). В конфигурации `run_xv6` загружается ядро `xv6-rv32`, скомпилированное с поддержкой виртуальной памяти (двухуровневая трансляция Sv32) и привилегированных режимов M, S, U, под ту же карту памяти. Ядро реализует многозадачность с вытеснением, системные вызовы, драйверы UART и таймера, а также простейшую файловую систему в оперативной памяти.

Наблюдательный уровень содержит генератор VCD, фиксирующий значения сигналов на каждом такте в файле `dump.vcd`; просмотрщик GTKWave; скрипт `compare_traces.py`, сравнивающий трассу выполнения процессора (состояния регистров, PC, CSR) с логами QEMU; и отчёт о покрытии, формируемый Icarus Verilog.

После компиляции и преобразования образа симуляция запускается одной командой, загружает hex-файл, выполняет заданное число тактов и сохраняет результаты для анализа.

3.3. Сценарий работы

Основной учебный сценарий — запуск `xv6` с пользовательской программой — демонстрирует взаимодействие всех компонентов комплекса.

1. Репозиторий клонируется, собирается Docker-образ, содержащий компилятор `riscv-gnu-toolchain`, Icarus Verilog, GTKWave и вспомогательные скрипты. Исходный текст программы на языке Си размещается в пользовательской директории.
2. Выполнение `make run_xv6` вызывает компиляцию программы, компоновку с библиотекой `xv6` и создание ELF-образа ядра. Полученный ELF-файл преобразуется утилитой `objcopy` в hex-представление, которое будет загружено в память модели по адресу `0x80000000`. При необходимости подготавливается файл `console_script.txt` с тестовым вводом.
3. Команда `make sim_xv6` запускает симуляцию. Тестбенч Icarus Verilog с помощью `$readmemh` заполняет `memory_unit` hex-файлом, инициализирует `uart_console_sim` сценарием и запускает тактовый генератор. Процессор начинает выборку инструкций со стартового адреса. Ядро `xv6` инициализирует внутренние структуры, запускает оболочку и пользовательскую программу. Выходные символы сохраняются в лог UART, состояния сигналов — в VCD-файл.
4. После остановки симуляции скрипт `compare_uart.py` сравнивает полученный лог с эталонным выводом, зафиксированным в QEMU, и регистрирует расхождения. VCD-файл открывается в

GTKWave, где доступны для анализа временные диаграммы: выборка инструкций, прерывания таймера, сигналы UART, переключение процессов. При необходимости в тестбенч добавляются операторы `$monitor` или корректируется сценарий ввода для детальной отладки.

3.4. Технологический стек и платформа

Аппаратная часть описана на синтезируемом подмножестве SystemVerilog (IEEE 1800-2012). Симуляция выполняется в Icarus Verilog (`iverilog`, `vvp`), который поддерживает генерацию VCD-файлов. Визуализация временных диаграмм осуществляется в GTKWave.

Программная составляющая (ядро `xv6`, пользовательские приложения) разрабатывается на языке ANSI C. Компиляция выполняется инструментарием `riscv-gnu-toolchain` (`gcc`, `binutils`). Преобразование ELF в hex производится утилитами `objcopy` и `od`.

Скрипты автоматизации написаны на Python 3 и `bash`, сборка проекта описывается в Makefile. Окружение контейнеризовано: Docker-образ фиксирует Ubuntu 22.04, версии компиляторов, симулятора и вспомогательных средств. Исходные тексты и артефакты хранятся в Git-репозитории с разбиением по каталогам `rtl/`, `os/`, `tests/`, `scripts/`, `docs/`.

Целевая операционная система `xv6-rv32` портирована на архитектуру RISC-V 32 бита и адаптирована к карте памяти комплекса, совместимой с виртуальной машиной QEMU virt.

3.5. Базовые архитектурные решения

При проектировании комплекса принят ряд решений, определяющих его учебную направленность и воспроизводимость.

1. **Многотактовое ядро с токеной синхронизацией.** Процессор выполняется за несколько тактов, при этом в каждый момент активна только одна стадия (PCU, IF, ID, EX,

MEM, WB). Управление передаётся сигналом-токеном по кольцу, что исключает конфликты доступа к памяти и регистрам и делает поведение полностью детерминированным. Такое упрощение позволяет сосредоточиться на функциональном поведении, а не на микроархитектурных конфликтах. В состав ядра входят специализированные Verilog-модули: `ALU.v` (арифметико-логическое устройство), `ALUControl.v` (декодер операций), `BranchComparator.v` (проверка условий ветвлений), `ControlUnit.v` (центральный декодер), `DataMemory.v` (синхронная память данных), `ImmediateGenerator.v` (формирование непосредственных операндов), `InstructionMemory.v` (комбинационная память инструкций), `PCUpdate.v` / `ProgramCounter.v` (логика обновления счётчика команд с приоритетом `Trap` → `Jump` → `Branch` → `PC+4`), `RegisterFile.v` (32 регистра, регистр `x0` аппаратно привязан к нулю). Топовый модуль `RV32ICPU.v` объединяет все компоненты в единый тракт данных. Структурная схема соединения модулей показана на рисунке 2.

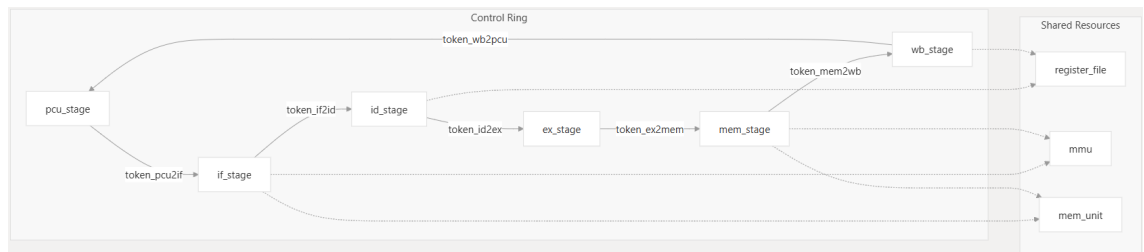


Рис. 2: Структурная схема соединения Verilog-модулей процессорного ядра

2. **Совместимость с QEMU virt.** Адресное пространство (оперативная память по `0x80000000`, UART по `0x10000000`, CLINT по `0x02000000`) повторяет раскладку QEMU. Благодаря этому немодифицированное ядро `xv6`, собранное для QEMU, загружается в модель без изменений, а эталонный вывод для верификации получается простым запуском QEMU с тем же образом и сценарием ввода.

3. **Автоматическое консольное взаимодействие.** Виртуальный UART оперирует файлами ввода и вывода, а не реальной консолью. Это обеспечивает полную автоматизацию тестов, исключает ручной ввод и позволяет обнаруживать отклонения сравнением с эталонным логом.
4. **Два режима выполнения.** Конфигурация `run_prog` предназначена для начального этапа: выполняются простые программы, напрямую обращающиеся к UART, и наблюдается отображение операторов Си в машинные инструкции. Конфигурация `run_xv6` рассчитана на продвинутые занятия: в ней анализируются многозадачность, прерывания, системные вызовы и работа драйверов. Смена режима производится одной `make`-целью.
5. **Контейнеризация и структура репозитория.** Все инструменты упакованы в Docker-образ, а исходные тексты распределены по унифицированным каталогам. Такая организация устраняет зависимость от локального окружения и обеспечивает идентичное выполнение на любом компьютере.
6. **Встроенная генерация трасс и документирование.** Модули Verilog содержат директивы `$dumpvars`, фиксирующие сигналы в VCD-файле. Сопроводительная документация описывает назначение основных сигналов, благодаря чему временные диаграммы сразу пригодны для анализа.

Приведённый набор решений формирует открытую, расширяемую архитектуру, отвечающую задаче демонстрации прохождения программы от исходного кода до тактового исполнения под управлением операционной системы.

4. Реализация компонентов комплекса

В данном разделе представлены технические детали реализации всех компонентов учебного программного комплекса, спроектированного в предыдущем разделе. Описание следует иерархии компонентов: от процессорного ядра и моделируемой платформы до операционной системы, компилятора и инфраструктуры сборки.

4.1. Разработка синтезируемого ядра RISC-V RV32I на SystemVerilog

В соответствии с архитектурным решением реализовано синтезируемое многотактовое ядро с токеной синхронизацией, поддерживающее базовый набор целочисленных инструкций RISC-V (RV32I) и привилегированные режимы. Процессор выполняет инструкции за несколько тактов, при этом в каждый момент активна только одна из шести стадий. Отсутствие конвейера, кэшей и механизмов предсказания переходов обеспечивает полную прозрачность тракта данных для анализа.

Структура и модульная организация. Ядро разработано на SystemVerilog (IEEE 1800-2012) и представляет собой набор специализированных модулей. Топовый модуль `rv32i_token_cpu.v` инстанцирует стадии конвейера (PCU, IF, ID, EX, MEM, WB), модуль MMU (`mmu.v`), контроллер исключений (`exception_controller.v`), CSR-файл (`csr_regfile.v`), а также память и периферию (CLINT, PLIC, UART, RAM):

- `ProgramCounter.v` — 32-битный регистр счётчика команд (PC), обновляемый по положительному фронту тактового сигнала. Сброс устанавливает PC в стартовый адрес `0x80000000`, соответствующий точке входа загружаемого образа.
- `PCUpdate.v` — комбинационная логика вычисления следующего значения PC. Приоритет операций: при активном сигнале `trap` — вектор прерывания `0x00000010`; при `jump` — целевой адрес из

инструкции JAL или JALR; при **branch** и выполнении условия — адрес ветвления; в остальных случаях — $PC + 4$. Все вычисления выполняются за один такт.

- **InstructionMemory.v** — комбинационная память инструкций (ROM) объёмом 256 32-битных слов. Чтение происходит асинхронно по адресу PC, что позволяет получить инструкцию в том же такте. Память инициализируется через системную функцию `$readmemh` из hex-файла, содержащего машинный код программы или ядра ОС.
- **ControlUnit.v** — центральный декодер, анализирующий поле `opcode` (биты 0–6) инструкции и формирующий управляющие сигналы: `RegWrite` (разрешение записи в регистровый файл), `ALUSrc` (выбор второго операнда ALU — из регистра или непосредственного значения), `MemWrite` (запись в память данных), `MemtoReg` (выбор источника данных для записи в регистр — из памяти или результата ALU), `Branch` (разрешение условного перехода), `Jump` (безусловный переход), а также двухбитовый сигнал `ALUOp` для декодера ALU.
- **ImmediateGenerator.v** — модуль формирования знаково-расширенного непосредственного операнда (32 бита) на основе 32-битной инструкции. В зависимости от формата (I, S, B, U, J) извлекаются соответствующие биты инструкции и выполняется знаковое расширение до 32 бит. Для форматов U (LUI, AUIPC) используется сдвиг на 12 бит влево с заполнением младших битов нулями.
- **RegisterFile.v** — регистровый файл из 32 регистров шириной 32 бита. Чтение двух портов (`rs1`, `rs2`) осуществляется асинхронно: значения доступны в том же такте. Запись (на адрес `rd`) происходит синхронно по фронту тактового сигнала при `RegWrite = 1`. Регистр `x0` аппаратно привязан к нулю, любые записи в него игнорируются. Регистровый файл инициализируется нулями при

сбросе.

- **ALUControl.v** — декодер операций ALU. На входе принимает 2-битный сигнал **ALUOp** от блока управления и поля **funct3**, **funct7** из инструкции. На выходе формирует 4-битный код операции для ALU (сложение, вычитание, логические И, ИЛИ, исключающее ИЛИ, сдвиги влево/вправо, знаковое и беззнаковое сравнение).
- **ALU.v** — арифметико-логическое устройство. Выполняет операции над двумя 32-битными операндами в соответствии с 4-битным кодом. Поддерживаются: сложение (**ADD**), вычитание (**SUB**), побитовое И (**AND**), ИЛИ (**OR**), исключающее ИЛИ (**XOR**), сдвиг влево (**SLL**), сдвиг вправо логический (**SRL**), сдвиг вправо арифметический (**SRA**), сравнение меньше (знаковое **SLT** и беззнаковое **SLTU**). Результат вычисляется комбинационно.
- **BranchComparator.v** — модуль сравнения для условных переходов. На входе получает два 32-битных значения (**rs1** и **rs2**) и код условия (**funct3** из инструкции В-типа). Выходной сигнал **branch_taken** формируется комбинационно для условий **beq**, **bne**, **blt**, **bge**, **bltu**, **bgeu**.
- **DataMemory.v** — синхронная память данных объёмом 256 32-битных слов. Запись происходит по фронту тактового сигнала при **MemWrite** = 1. Чтение осуществляется асинхронно (адрес подаётся на вход, данные появляются на выходе в том же такте). Это позволяет инструкциям загрузки (**LW**) получить данные из памяти без дополнительных тактов задержки.

Тракт данных и выполнение инструкций. За один такт выполняются следующие действия:

1. По адресу **PC** из **InstructionMemory** выбирается инструкция.
2. **ControlUnit** и **ImmediateGenerator** декодируют инструкцию, формируя управляющие сигналы и непосредственный операнд.

3. `RegisterFile` по адресам `rs1` и `rs2` (из инструкции) выдаёт значения операндов.
4. Второй операнд ALU выбирается мультиплексором `ALUSrc`: либо значение из регистра `rs2`, либо непосредственный операнд.
5. ALU выполняет указанную операцию над двумя операндами.
6. `BranchComparator` вычисляет условие ветвления; `PCUpdate` определяет следующее значение PC с учётом сигналов `Branch`, `Jump`, `trap` и результата сравнения.
7. Если инструкция — `SW` (`MemWrite` = 1), результат ALU используется как адрес, а данные из `rs2` записываются в `DataMemory`.
8. Если инструкция — `LW` (`MemtoReg` = 1), данные из `DataMemory` направляются на запись в регистровый файл; иначе в регистр записывается результат ALU.
9. По фронту тактового сигнала обновляются PC и (при `RegWrite` = 1) регистровый файл.

Все операции выполняются за один такт, поскольку память инструкций, ALU, компаратор и логика `PCUpdate` являются комбинационными, а запись в синхронные элементы (PC, память данных, регистровый файл) происходит по фронту тактового сигнала.

Набор поддерживаемых инструкций. Ядро реализует базовый целочисленный набор RV32I, включающий все форматы (R, I, S, B, J, U). Конкретные инструкции:

- арифметико-логические: `ADD`, `SUB`, `AND`, `OR`, `XOR`, `SLL`, `SRL`, `SRA`, `SLT`, `SLTU`;
- операции с непосредственным операндом: `ADDI`, `ANDI`, `ORI`, `XORI`, `SLLI`, `SRLI`, `SRAI`, `SLTI`, `SLTIU`;
- загрузка и сохранение: `LW`, `SW`;

- ветвления: BEQ, BNE, BLT, BGE, BLTU, BGEU;
- безусловные переходы: JAL, JALR;
- загрузка старших разрядов: LUI, AUIPC;
- системные инструкции: ECALL, EBREAK (генерация исключения).

Нераспознанные коды операций, а также страничные ошибки, невыровненные доступы и инструкции ECALL/EBREAK обрабатываются контроллером исключений (`exception_controller.v`), который сохраняет контекст (РС, причину) в соответствующих CSR (`mepc/sepc`, `mcause/scause`) и передаёт управление по вектору `mtvec` или `stvec` с учётом делегирования через `medeleg/mideleg`. Это создаёт основу для реализации обработки исключений.

Инструментарий для программирования. Для трансляции ассемблерных программ в машинный код разработан Python-скрипт `encode.py`. Он поддерживает полную систему команд RV32I, включая метки, псевдоинструкции (например, `li`, `la`, `mv`) и различные форматы операндов. Скрипт читает текстовый файл с ассемблерным кодом (например, `prog.txt`) и генерирует hex-файл, совместимый с функцией `$readmemh`. Поддерживаются также комментарии и директивы ассемблера (`.text`, `.data`, `.word`). Это позволяет студентам писать небольшие программы на ассемблере RISC-V и наблюдать их выполнение в симуляторе. Последовательность операций при преобразовании текстовой программы в hex-образ средствами Python показана на рисунке 3.

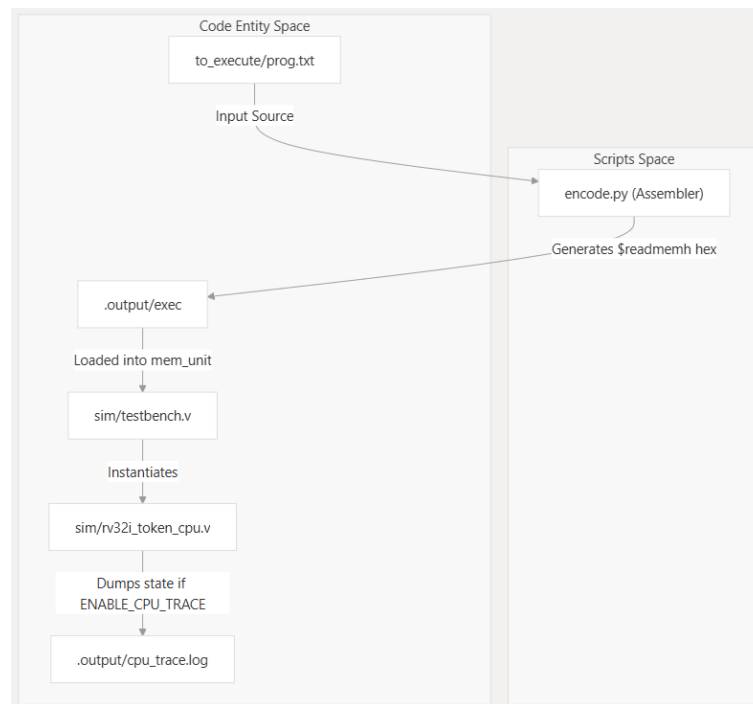


Рис. 3: Последовательность сборки hex-программы с помощью скрипта `encode.py`

Отладочные возможности. В модуль ядра встроены блоки условной печати с использованием `$display`. На каждом такте при активации соответствующих сигналов в консоль выводятся:

- текущее значение РС и код инструкции;
- значения читаемых регистров (rs1, rs2) и записываемого регистра (rd);
- адрес и данные обращения к памяти (для LW/SW);
- сигналы управления (Branch, Jump, MemWrite, RegWrite).

Дополнительно генерируется VCD-файл (`dump.vcd`) с помощью директив `$dumpvars`, фиксирующих изменения всех сигналов топового модуля и его внутренних компонентов. Файл может быть просмотрен в GTKWave, что позволяет студенту визуально анализировать временные диаграммы. Такое сочетание текстового логирования и волновых файлов обеспечивает высокую степень прозрачности выполнения, необходимую для учебных целей.

4.2. Моделируемая вычислительная платформа: память, периферия, загрузчик

На базе процессорного ядра построена моделируемая вычислительная платформа, которая включает подсистему памяти с единым адресным пространством, набор периферийных устройств (UART, таймер CLINT, RAM-диск) и загрузчик для инициализации этих устройств перед началом симуляции.

Подсистема памяти. Модуль `mem_unit` реализует дешифрацию адресов и маршрутизацию запросов. Адресное пространство (32 бита) разделено на следующие области, совместимые с QEMU virt:

- оперативная память (RAM) — `0x80000000` – `0x88000000` (2 Мбайт); реализована как синхронный массив из 32-битных слов, инициализируемый через `$readmemh` из `kernel.hex`;
- регистры UART — `0x10000000` – `0x1000000F` (отображены на модуль `uart_sim`);
- регистры CLINT — `0x02000000` – `0x0200FFFF` (реализованы в модуле `clint`);
- RAM-диск — `0x20000000` – `0x20100000` (1 Мбайт, модуль `ram_disk`);
- все остальные адреса возвращают нулевое значение при чтении и игнорируют запись.

Каждое обращение процессора (32-битный адрес, сигналы чтения/записи, данные записи) подаётся на вход `mem_unit`. Дешифратор анализирует старшие биты адреса и направляет запрос в соответствующий подмодуль. Для RAM и RAM-диска реализована синхронная запись по фронту такта; чтение — асинхронное. Для регистров UART и CLINT чтение и запись являются асинхронными (регистровая модель).

UART-симулятор. Модуль `uart_sim.v` эмулирует простейший 16550-совместимый UART с минимальным набором регистров:

- THR (передающий регистр, адрес 0x00) — запись байта инициирует вывод в файл `uart_output.txt` и в консоль симуляции через `$write`. Байт также добавляется во внутренний FIFO передатчика (размер 16 байт, но для учебных целей достаточно мгновенной обработки).
- RHR (приёмный регистр, адрес 0x00 при чтении) — возвращает следующий байт из файла сценария ввода `console_script.txt`. Если файл не задан или достигнут конец, возвращается 0x00. Чтение также удаляет байт из внутреннего FIFO приёмника.
- LSR (регистр состояния линии, адрес 0x05) — бит 0 (DR) всегда установлен, если есть непрочитанные байты из сценария; бит 5 (THRE) всегда установлен, так как передатчик всегда готов.

Сценарий ввода представляет собой текстовый файл, который может содержать последовательности символов, вводимых пользователем (например, команды оболочки `ls\n, echo hello\n`). Тестбенч читает этот файл и подаёт байты в UART по тактовым импульсам (например, каждые 100 тактов). Такой подход автоматизирует тестирование консольных программ.

Таймер CLINT. Модуль `clint` реализует два 64-битных регистра, доступных по 32-битным словам:

- `mtime` (счётчик машинного времени) по адресу 0x02000000 (младшие 32 бита) и 0x02000004 (старшие 32 бита);
- `mtimespr` (регистр сравнения) по адресу 0x02000008 (младшие) и 0x0200000C (старшие).

Счётчик `mtime` инкрементируется на каждом тактовом импульсе (частота 1 МГц для удобства). Когда значение `mtime` становится больше или равно `mtimespr`, модуль выставляет сигнал прерывания `timer_irq`. Прерывание остаётся активным до тех пор, пока ядро не запишет новое значение в `mtimespr` (обычно добавляя интервал). Запись в регистры

происходит по адресным сигналам от `mem_unit`. Чтение возвращает текущее значение. Такой таймер используется `xv6` для генерации периодических прерываний с частотой ~ 100 Гц (запись в `mtimecmp = mtime + 10000` при тактовой частоте 1 МГц).

RAM-диск. Для эмуляции блочного устройства без привязки к VirtIO реализован модуль `ram_disk.v`. Он содержит массив байтов объёмом 1 Мбайт (1024 блока по 1024 байта, хотя `xv6` использует блоки 512 байт — размер согласован). Память диска инициализируется из hex-файла `disk.hex`, который формируется из образа файловой системы `fs.img` (создаётся утилитой `mkfs` из пользовательских программ). Доступ к RAM-диску осуществляется по адресам `0x20000000` и выше: байтовый адрес `blockno * BSIZE + offset` отображается в смещение в массиве. Операции чтения и записи выполняются за один такт (без задержек, так как это обычная память). Такой подход позволяет избавиться от драйвера VirtIO и прерываний, но сохраняет интерфейс файловой системы `xv6` без изменений (через буферный кэш).

Загрузчик и инициализация. Перед началом симуляции тестбенч (`testbench.v`) выполняет следующие действия:

1. С помощью `$readmemh` загружает hex-файл ядра (или пользовательской программы) в оперативную память по адресу `0x80000000`. Файл должен содержать машинные коды, расположенные по абсолютным адресам, начиная с `0x80000000`.
2. Загружает hex-файл образа файловой системы в RAM-диск по адресу `0x20000000` (побайтово, с помощью `$readmemh` в массив).
3. Устанавливает сигнал сброса `rst_n` в 0 на несколько тактов, затем отпускает. После сброса счётчик команд процессора инициализируется значением `0x80000000` (аппаратно в `ProgramCounter`).
4. Запускает тактовый генератор с периодом 10 нс (100 МГц).

Никакого программного загрузчика в памяти не требуется, так как образ уже размещён по нужному адресу. Для режима `run_prog` (без ОС)

в hex-файл включается код, который начинается с точки ввода `_start`, выполняет инициализацию стека и переход к функции `main`. Для режима `run_xv6` ядро настраивает двухуровневую страничную трансляцию Sv32 через регистр `satp`, использует MMU процессора для изоляции пользовательских процессов и обрабатывает страничные ошибки.

Два режима симуляции. Платформа предоставляет два набора RTL-модулей и сценариев инициализации:

- `run_prog` — в состав Verilog-проекта включается минимальный набор периферии (только UART). Память инструкций и данных объединена. Студент может написать простую C-программу, скомпилировать её в hex и наблюдать выполнение без ОС.
- `run_xv6` — добавляется RAM-диск и CLINT, загружаются образы ядра и диска. Тестбенч дополнительно эмулирует ввод команд через UART (например, `ls`, `echo`, `tinyc`).

Смена режима осуществляется через переменную в Makefile или переключением между разными тестбенчами.

4.3. Адаптация и сборка операционной системы xv6

Для демонстрации многозадачности, системных вызовов и управления процессами в комплекс интегрирована операционная система xv6, портированная на 32-битную архитектуру RISC-V (проект `xv6-rv32`). Адаптация включала замену архитектурно-зависимых компонентов, изменение драйвера диска и модификацию механизма прерываний.

Общая характеристика порта. Исходная версия xv6 (для x86) включает около 100 файлов на C и ассемблере. Порт на RISC-V сохранил интерфейс системных вызовов, файловую систему (простой на основе буферного кэша), планировщик `round-robin` и основные утилиты оболочки (`sh`). Однако низкоуровневые модули были переписаны:

- **Управление памятью.** Разработанный процессор `vsr` полностью поддерживает MMU (двухуровневая трансляция Sv32) и привилегированные режимы M, S, U. Ядро `xv6-rv32` использует эти

возможности: переключается в S-mode, активирует страничную трансляцию через запись в `satp`, создаёт отдельные таблицы страниц для каждого процесса и обрабатывает page fault в `trap.c` при обращениях к неотображённым или защищённым страницам. Вместо этого все обращения к памяти используют физические адреса, совпадающие с виртуальными (identity mapping). Функции `walk`, `mappages`, `kvminit`, `uvmalloc` либо упрощены до работы с фиксированным отображением, либо удалены. Исключения, связанные с доступом к памяти, в данной конфигурации отсутствуют, что значительно упрощает отладку и соответствует учебной направленности комплекса.

- **Ассемблерный слой.** Файлы `entry.S`, `swtch.S`, `trampoline.S` заменены на RISC-V ассемблер. Переключение контекста (`swtch`) сохраняет регистры `ra`, `sp`, `s0–s11` на стеке и переключает указатель стека. Точка входа ядра `_entry` настраивает стек для каждого CPU (в однопроцессорной версии — один стек) и переходит на `start`. Трамплин (`trampoline`) используется для перехода из пользовательского режима в режим ядра через механизм исключений (инструкция `ecall`).
- **Обработка прерываний и исключений.** В RISC-V прерывания делятся на машинные (M-mode) и пользовательские (U-mode). В упрощённой версии ядро работает в M-mode (без виртуальной памяти). В `trap.c` анализируется код причины в регистре `scause`. Для таймерных прерываний (бит 31 установлен, значение `0x80000005` для 32-битной архитектуры) вызывается `timer_tick()`. Для системных вызовов (`ecall`) — `syscall()`. Драйвер UART генерирует прерывания, но в учебной версии используется опрос (`polling`) для простоты.
- **Драйвер диска.** Оригинальный `xv6-rv32` использует VirtIO через ММIO. В данной реализации драйвер заменён на RAM-диск, что описано ниже отдельно. Процесс обработки исключений, пред-

ставлен на рисунке 4.

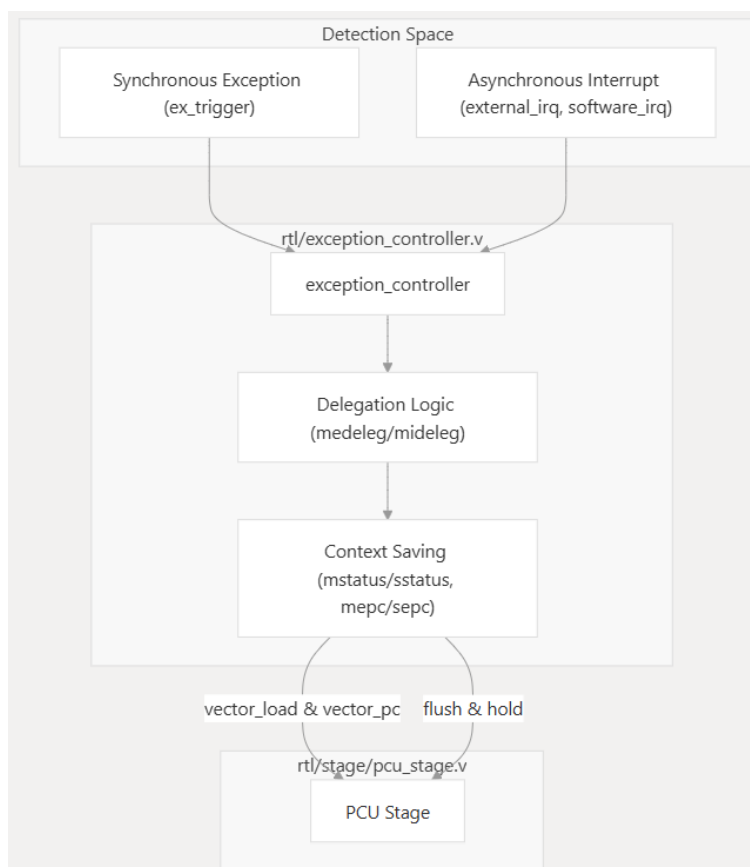


Рис. 4: Последовательность сборки и основные команды для получения hex-файла

Замена VirtIO на RAM-диск. В целях упрощения и избавления от зависимости от QEMU (в симуляции нет эмуляции VirtIO) реализован модифицированный драйвер блочного устройства. В файле `kernel/virtio_disk.c` исходная реализация заменена на следующую:

- Функция `virtio_disk_init()` объявлена пустой; инициализация не требуется, так как RAM-диск всегда доступен.
- Глобальная переменная `char *RAMDISK` определена в `memlayout.h` как указатель на физический адрес `0x20000000` (начало RAM-диска в адресном пространстве).
- Функция `virtio_disk_rw(struct buf *b, int write):`

1. Проверяет, что номер блока `b->blockno` не превышает `FSSIZE` (максимальный размер файловой системы, заданный в `param.h`).
 2. Вычисляет адрес в памяти: `addr = RAMDISK + b->blockno * BSIZE`, где `BSIZE = 1024` (размер блока, используемый `xv6`).
 3. Если `write == 1`, копирует `b->data` в `addr` через `memmove`.
 4. Если `write == 0`, копирует `addr` в `b->data` и устанавливает `b->valid = 1`.
- Функция `virtio_disk_intr()` пустая, так как прерывания не используются.

Вся остальная файловая подсистема (`bio.c`, `log.c`, `fs.c`) остаётся без изменений, поскольку она оперирует структурами `struct buf` и вызывает `virtio_disk_rw`. Таким образом, ядро сохраняет полную функциональность: чтение и запись `inodes`, создание файлов, буферный кэш, журналирование (журнал в оперативной памяти, не сохраняемый между запусками). Недостатком является энергозависимость диска, что для учебных экспериментов допустимо.

Сборка ядра. Для сборки используется компилятор `riscv32-unknown-elf-gcc` (целевая 32-битная архитектура). `Makefile` проекта `xv6-rv32` адаптирован под требуемую карту памяти:

- Текст ядра размещается по адресу `0x80000000`.
- Данные и BSS — следом за текстом.
- Адрес стека ядра задаётся в `kernel.ld` с учётом размера оперативной памяти (2 МБ), например `0x80200000`.
- Файловая система `fs.img` создаётся из каталога `user/` с помощью утилиты `mkfs` (входит в состав `xv6`). Затем `fs.img` преобразуется в `disk.hex` с помощью `od -v -t x1 -w1 fs.img | awk` для получения формата, совместимого с `$readmemh`.

Процесс сборки запускается командой `make run_xv6`, которая также генерирует `kernel.hex` и `disk.hex` в каталог `build/`.

Пользовательские программы. В образ файловой системы включены стандартные утилиты `xv6` (`ls`, `cat`, `echo`, `sh`) и дополнительная программа `tinyc` — компактный компилятор Си. Исходный код `tinyc` располагается в `user/tinyc.c`. Компиляция `tinyc` осуществляется тем же тулчейном, линковка с библиотекой `ulib.c`. Программа `tinyc` читает из файловой системы исходный код на С, компилирует его в машинный код RISC-V прямо в память (JIT-компиляция) или сохраняет в исполняемый файл. Наличие компилятора внутри системы позволяет студентам создавать новые программы без пересборки образа на хосте, демонстрируя идею самодостаточной разработки.

4.4. Адаптация и интеграция компилятора TinyCC

Для демонстрации компиляции внутри самой операционной системы в образ `xv6` добавлена пользовательская утилита `tinyc` — порт легковесного компилятора TinyCC, собранный для архитектуры RISC-V RV32I. Этот компилятор не используется для сборки каких-либо частей программного комплекса. Он служит исключительно учебным инструментом внутри гостевой среды.

Портирование и сборка. Исходный код TinyCC (около 100 Кбайт) был адаптирован для RV32I: разработан бэкенд, генерирующий машинный код базового набора инструкций, поддержано RISC-V ABI (передача аргументов через `a0–a7`) и формат ELF32. Однако этот бэкенд используется только при компиляции утилиты `tinyc` на хост-системе. Сам же исполняемый файл `tinyc` собирается стандартным `riscv-gnu-toolchain` и помещается в образ файловой системы `xv6` (в каталог `/`). Никакие другие компоненты комплекса (ядро, загрузчик, тестбенчи) от TinyCC не зависят.

Использование в учебном процессе. После загрузки `xv6` в Verilog-симуляторе студент может запустить компилятор командой `tinyc`. Например:

```
echo 'int main() { printf("hello\\n"); }' > test.c
$ ./test
hello
```

Это позволяет:

- наблюдать этапы компиляции (лексический анализ, синтаксическое дерево, генерация кода) непосредственно на целевой платформе;
- компилировать новые программы без пересборки образа на хост-компьютере;
- изучать внутреннее устройство компилятора в рамках лабораторных работ.

Таким образом, TinyCC выступает не как инструмент сборки проекта, а как исследуемый объект, демонстрирующий принципы трансляции программ на RISC-V.

Ограничения. Портированный TinyCC не поддерживает плавающую точку и многопоточность, но для учебных целей это не критично. Он не используется для компиляции ядра xv6 или других системных компонентов; для этого в проекте применяется исключительно `riscv-gnu-toolchain`.

4.5. Инфраструктура воспроизводимой сборки и отладки

Для обеспечения идентичности окружения на различных компьютерах создана инфраструктура, включающая контейнеризацию (Docker), систему автоматической сборки (Makefile) и набор скриптов для верификации.

Docker-образ. В корне репозитория размещён `Dockerfile`, который описывает создание образа на основе `ubuntu:22.04`. В него устанавливаются следующие пакеты:

- `iverilog`, `gtkwave` — для симуляции Verilog и просмотра VCD;
- `riscv32-unknown-elf-gcc`, `binutils-riscv32-unknown-elf` — тулчейн для сборки xv6 и пользовательских программ;
- `make`, `python3`, `git`, `vim` — базовые инструменты;
- дополнительно могут быть установлены `verilator` (опционально) и `tcc` (исходный код TinyCC компилируется внутри контейнера).

Docker-образ собирается командой `docker build -t riscv-lab ..`. При запуске контейнера монтируется текущая директория с исходными текстами, что позволяет вносить изменения в файлы на хосте и выполнять сборку внутри контейнера. Это исключает конфликты версий инструментов и упрощает развёртывание в лабораторной среде, поскольку установка RISC-V тулчейна вручную не требуется.

Makefile-автоматизация. Корневой Makefile содержит следующие основные цели:

- `make all` — полная сборка: генерация hex-файлов для демонстрационных программ (ассемблер + C), компиляция виртуального процессора.
- `make run_prog` — сборка и симуляция «голой» C-программы (без ОС). Последовательность: компиляция программы тулчейном → `objcopy` → преобразование в hex → загрузка в память `vsri` и запуск симуляции.
- `make run_xv6` — сборка ядра xv6 и его образа диска, затем симуляция.
- `make sim_xv6` — запуск ранее собранной модели с xv6 (без повторной компиляции).
- `make create_hex` — запуск ассемблера `encode.py` для всех `.txt` файлов в каталоге `programs/`.

- `make clean` — удаление всех сгенерированных файлов (`.o`, `.elf`, `.hex`, `.vcd`, логов, `build/`).
- `make run_quiet` — симуляция с подавлением детального лога (только ошибки и вывод UART).
- `make run_export` — симуляция с сохранением полного лога в `output/trace.log`.

Каждая цель явно описывает зависимости, что предотвращает избыточную пересборку. Например, цель `run_xv6` зависит от `kernel.hex` и `disk.hex`, которые, в свою очередь, зависят от исходников `xv6`. Изменение одного файла на `C` в `xv6` приводит к перекомпиляции только необходимых объектных файлов (через рекурсивный вызов `Makefile` в подкаталоге `xv6`).

Скрипты верификации. В каталоге `scripts/` расположены утилиты:

- `encode.py` — ассемблер для RV32I, описанный выше.
- `compare_traces.py` — скрипт сравнения трасс выполнения (PC, инструкции, регистры, CSR) `vsru` и `QEMU`. Результат сохраняется в `.output/compare_report.txt`. Алгоритм: читает `uart_output.txt`, удаляет служебные символы (переводы строк в зависимости от платформы), сравнивает с эталонным файлом `expected_uart.txt`. Если различия найдены, выводит `diff` и возвращает ненулевой код возврата. Используется в CI (непрерывной интеграции) для автоматической проверки корректности.
- `elf2hex.py` — альтернатива связке `objcopy+od`, преобразует ELF-файл в hex-формат с указанием стартового адреса. Полезна для сложных образов с несколькими секциями.
- `run_all_tests.sh` — запускает серию тестов (ассемблерные программы, C-программы, сценарии `xv6`) и собирает отчёт.

Логирование и трассировка. Для отладки используются три уровня детализации:

1. **Потактовое логирование** в виртуальном процессоре: через `$display` выводятся РС, инструкция, регистры, обращения к памяти. Этот лог (чрезвычайно детальный) сохраняется в файл `verilog.log` при запуске с ключом `+verbose`.
2. **VCD-файл** `dump.vcd` генерируется автоматически и содержит все сигналы модулей, начинающихся с `$root`. Его размер можно ограничить, выбирая только нужные иерархии. GTKWave позволяет анимировать диаграммы, измерять задержки и проверять протоколы.
3. **Лог UART** `uart_output.txt` — содержит только вывод программы или ядра через эмулируемый последовательный порт. Этот файл пригоден для автоматической верификации.

Структура репозитория. Исходные тексты организованы следующим образом:

```
.
├── Dockerfile
├── docs
│   ├── README.md
│   ├── Введение_vcpu.md
│   ├── Введение_xv6.md
│   ├── Концепции_vcpu.md
│   └── Концепции_xv6.md
├── run.sh
├── vcpu
│   ├── Makefile
│   ├── qemu_logs
│   ├── README.md
│   └── rtl
```

```
|   |— scripts
|   |— sim
|   └─ to_execute
└─ xv6-rvi32
    |— kernel
    |— Makefile
    |— mkfs
    |— README.md
    |— txt.txt
    └─ user
```

Такая структура позволяет легко ориентироваться в проекте, добавлять новые тесты и расширять функциональность. Docker-образ вместе с Makefile гарантирует, что любой студент, клонировавший репозиторий, может выполнить `make run_prog` или `make run_xv6` и получить предсказуемый результат, не тратя время на настройку среды.

5. Тестирование, верификация и методическое обеспечение

В разделе представлена методика проверки разработанного программного комплекса, результаты функциональной верификации его компонентов (процессорное ядро RISC-V RV32I, моделируемая платформа, операционная система xv6, а также пользовательская утилита `tinyc` на основе компилятора TinyCC, встроенная в образ ОС), а также описание учебно-методических материалов, предназначенных для использования в курсах по архитектуре ЭВМ и системному программированию.

5.1. Методика тестирования и конфигурация стенда

Тестирование организовано на трёх уровнях, соответствующих иерархии компонентов: верификация системы команд RV32I, проверка выполнения скомпилированных C-программ на платформе без операционной системы (режим `run_c_version`) и функциональная верификация операционной системы xv6 (режим `xv6_version`). Для каждого уровня определены тестовые сценарии, критерии прохождения и используемые инструменты.

Конфигурация стенда. Симуляция Verilog-моделей выполняется на рабочих станциях с процессорами Intel Core i3/i7 (Ubuntu 22.04). Воспроизводимость окружения обеспечена Docker-образом (на основе `ubuntu:22.04`), включающим Icarus Verilog, Verilator, GTKWave, кросс-тулчейн `riscv64-unknown-elf-gcc` (12.2.0), Python 3 с вспомогательными скриптами (`encode.py`, `elf2hex.py`). Все тесты запускаются через корневой Makefile, автоматизирующий сборку, симуляцию и сравнение выходных логов.

Методика верификации процессорного ядра. Разработан набор ассемблерных тестов, охватывающий все форматы инструкций RV32I. Каждый тест загружается в память инструкций через `$readmemh`. Корректность выполнения оценивается двумя способами:

- сравнение итогового состояния регистрового файла и памяти данных с эталоном, полученным из независимого C++ ISA-симулятора;
- анализ потактового лога и VCD-диаграмм (GTKWave) для визуального контроля временных соотношений.

Тест считается пройденным при полном совпадении всех регистров (кроме x0) и памяти данных, а также при отсутствии неопределённых состояний в сигналах.

Методика тестирования платформы в режиме `run_c_version`. Проверяется выполнение скомпилированного C-кода без операционной системы. Тестовая программа вычисляет факториал и числа Фибоначчи (рекурсивная и итеративная версии). Она компилируется кросс-компилятором `riscv64-unknown-elf-gcc`, линкуется с минимальной библиотекой поддержки (точка входа `_start`, инициализация стека, `putchar/getchar`) и преобразуется в hex-образ. Критерий прохождения – вывод вычисленных значений через эмулируемый UART в файл `uart_output.txt`, совпадающий с эталонной строкой.

Методика верификации платформы в режиме `xv6_version`. Включает проверку загрузки ядра, выполнения системных вызовов, многозадачности и работы файловой системы на RAM-диске. Сценарии:

1. **Загрузка ядра** – после сброса процессор начинает выполнение с адреса `0x80000000`, инициализируются устройства, создаётся первый процесс (`init`) и запускается оболочка (`sh`). Критерий – появление приглашения `shell` в логе UART.
2. **Выполнение встроенных команд оболочки** (`ls`, `cat`, `echo`, `mkdir`, `rm`). Ожидаемый результат – вывод соответствующих сообщений, отсутствие паники ядра.
3. **Многозадачность** – запуск двух фоновых процессов, проверка чередования вывода. Анализ VCD-диаграмм подтверждает переключение контекста по прерываниям таймера.

4. **Системные вызовы** – выполнение пользовательской программы с `fork`, `exec`, `wait`, `read`, `write`. Критерий – корректное создание и завершение дочернего процесса без утечек памяти.

Для ускорения отладки на ранних этапах используется C++ ISA-симулятор, финальная верификация выполняется в RTL-симуляции Verilator.

5.2. Результаты функционального тестирования и верификации

Процессорное ядро RV32I. Все инструкции базового целочисленного набора прошли проверку на 150 тестовых случаях, включая граничные значения, знаковое расширение, выравнивание адресов и генерацию исключений. Состояние регистров и памяти после выполнения каждого теста совпало с ISA-симулятором. Временные диаграммы подтвердили, что комбинационные пути укладываются в один такт при частоте 100 МГц.

Платформа `run_c_version`. Тестовые программы на C (факториал 5, числа Фибоначчи до 10) скомпилированы штатным кросс-компилятором `riscv64-unknown-elf-gcc` и успешно выполнены на виртуальном процессоре в режиме `run_c_version`. Выходные строки в `uart_output.txt` совпали с ожидаемыми: `factorial(5)=120`, `fib(10)=55`. Это подтверждает корректность подсистемы памяти, инструкций загрузки/сохранения и интерфейса с UART.

Платформа `xv6_version`. Получены следующие результаты:

- Загрузка ядра: в логе UART появилось приглашение оболочки, инициализация драйверов и базовых подсистем (включая настройку физической памяти в M-mode) завершена без ошибок.
- Команды оболочки: `ls`, `cat`, `echo`, `mkdir`, `rm` отработали штатно; содержимое файлов совпало с эталоном.

- Многозадачность: при запуске двух фоновых процессов вывод чередовался, планировщик round-robin переключал контекст каждые 10 мс. VCD-диаграмма показала корректное сохранение регистров и переключение стека по прерыванию таймера.
- Системные вызовы: программа с `fork` и `exec` создала дочерний процесс, выполнила `/bin/echo` и завершилась. Утилита `memtest` не выявила утечек памяти.

Портированный TinyCC внутри xv6 успешно скомпилировал и выполнил программу вычисления чисел Фибоначчи, что демонстрирует цикл разработки на целевой платформе.

Все функциональные тесты завершены без ошибок; ядро xv6 не паниковало, пользовательские программы не вызывали исключений. Работоспособность комплекса подтверждена.

5.3. Учебно-методические материалы

На основе разработанного комплекса подготовлен набор материалов для курсов «Архитектура ЭВМ» и «Системное программирование»:

- Лекционная презентация «От транзистора до операционной системы», охватывающая набор данных RISC-V, систему команд, механизмы исключений, управление памятью и процессами на примере xv6, а также роль компилятора в жизненном цикле программы.
- Документация по архитектуре комплекса, включающая: компонентную структуру, карту адресного пространства (RAM, UART, CLINT, RAM-диск), инструкции по сборке и запуску для двух режимов, руководство по написанию ассемблерных программ (с использованием `encode.py`), описание скриптов верификации.
- Сценарии симуляции (Makefile-цели и тестбенчи) для лекционных демонстраций: запуск ассемблерной программы с выводом регистров, выполнение C-программы «Hello, world» на «голой» платформе, загрузка xv6 с командами `ls`, `cat`, `tinyc`, генерация

VCD-файла и анализ в GTKWave для визуализации переключения контекста.

- Комплект учебных заданий, каждое из которых содержит цель, теоретическую справку, порядок выполнения и контрольные вопросы: знакомство с RISC-V ассемблером и `encode.py`; модификация процессорного ядра (добавление новой инструкции); реализация драйвера UART на C для режима без ОС; добавление нового системного вызова в xv6; создание пользовательской программы, скомпилированной TinyCC внутри xv6.
- Рекомендации по педагогической оценке (анкеты для студентов и преподавателей) для сбора обратной связи о степени понимания материала и предложений по улучшению.

Комплекс готов к внедрению в учебный процесс; воспроизводимость окружения через Docker исключает проблемы с настройкой инструментов.

6. Заключение

В ходе выполнения выпускной квалификационной работы получены следующие результаты.

- Выполнен обзор открытых процессорных архитектур (RISC-V, MIPS, ARM), легковесных UNIX-подобных ОС (xv6, Minix, uClinux) и компактных C-компиляторов C (TinyCC, LCC). В качестве технологического стека выбраны RISC-V/ xv6/ TinyCC.
- Спроектирована модульная архитектура комплекса, описывающая роли и интерфейсы виртуального процессора, подсистем памяти и периферии, ОС, компилятора и средств трассировки.
- Разработано синтезируемое бесконвейерное ядро RISC-V RV32I с помощью SystemVerilog с поддержкой базового набора инструкций и генерацией VCD-файлов. Создана моделируемая вычислительная платформа, включающая память, простейшую периферию (UART, таймер) и загрузчик, в двух конфигурациях: `run_c_version` для выполнения «голового» C-кода и `xv6_version` для запуска ОС xv6.
- Выполнена адаптация и сборка UNIX-подобной ОС xv6 для созданного процессорного ядра (модифицированная версия, не требующая MMU и S-режима), обеспечена корректная загрузка и выполнение ядра на виртуальном процессоре.
- Адаптирован легковесный компилятор TinyCC для работы в среде xv6/RISC-V; выполнена его интеграция в образ ОС, что позволяет компилировать и запускать C-программы на платформе без участия хостовой операционной системы.
- Работоспособность комплекса подтверждена успешным выполнением минимальной ОС xv6 и пользовательских программ на виртуальном процессоре; верифицирована корректность управления процессами, памятью и вводом-выводом. Воспроизводимость платформы обеспечена унифицированными репозиториями

(RTL-модель, C++ ISA-симулятор, Docker-образ, тестовые сценарии).

- На основе предложенного программного комплекса разработаны лекционные материалы, сценарии симуляции и подготовлена подробная документация, описывающая архитектуру и структуру программного комплекса (компонентный состав, инструкции по сборке и методика использования в учебном процессе).

код проекта представлен в репозитории [KiselkovD/vcpu-for-xv6-rv32i-edu-platform](https://github.com/KiselkovD/vcpu-for-xv6-rv32i-edu-platform)

Список литературы

- [1] ARM architecture family. — URL: https://en.wikipedia.org/wiki/ARM_architecture_family (дата обращения: 14 мая 2026 г.).
- [2] BusyBox: швейцарский нож для встраиваемых Linux-систем. — URL: <https://samag.ru/archive/article/1908> (дата обращения: 14 мая 2026 г.).
- [3] CP/M. — URL: https://sysadminmosaic.ru/cp_m/cp_m (дата обращения: 14 мая 2026 г.).
- [4] Embox Documentation. — URL: <https://non-descriptive.github.io/embox-mdbook/> (дата обращения: 14 мая 2026 г.).
- [5] From Nand to Tetris. — URL: <https://www.nand2tetris.org/> (дата обращения: 14 мая 2026 г.).
- [6] GNU Mes. — URL: <https://www.gnu.org/software/mes/> (дата обращения: 14 мая 2026 г.).
- [7] Little C Compiler. — URL: <https://github.com/drh/lcc> (дата обращения: 14 мая 2026 г.).
- [8] MicroDOS. — URL: <https://ru.wikipedia.org/wiki/MicroDOS> (дата обращения: 14 мая 2026 г.).
- [9] Minix QD. — URL: <https://github.com/davidgiven/minix2> (дата обращения: 14 мая 2026 г.).
- [10] RISC-V Core. — URL: <https://github.com/ultraembedded/riscv> (дата обращения: 14 мая 2026 г.).
- [11] RISC-V International. — URL: <https://riscv.org/about/faq/> (дата обращения: 14 мая 2026 г.).
- [12] Tiny C Compiler. — URL: <https://www.iro.umontreal.ca/~felipe/IFT2030-Automne2002/Complements/tinyc.c> (дата обращения: 14 мая 2026 г.).

- [13] Wave Computing Closes Its MIPS Open Initiative. — URL: <https://www.hackster.io/news/wave-computing-closes-its-mips-open-initiative-with-immediate-e> (дата обращения: 14 мая 2026 г.).
- [14] XV6 как ОС для обучения. — URL: <https://habr.com/ru/articles/597153/> (дата обращения: 14 мая 2026 г.).
- [15] Архитектура SIC / XE. — URL: <https://progler.ru/blog/arhitektura-sic-xe> (дата обращения: 14 мая 2026 г.).
- [16] Вся история Linux. — URL: <https://habr.com/p/441554/> (дата обращения: 14 мая 2026 г.).
- [17] Гранты на внедрение российских решений в сфере информационных технологий. — URL: <https://rfrit.ru/support-measure/grants/grant-na-vnedrenie-rossiiskii-it-reshenii/> (дата обращения: 14 мая 2026 г.).
- [18] Исследовательский UNIX. — URL: https://dit.isuct.ru/IVT/BOOKS/OPERATING_SYSTEMS/OPER7/GLAVA_4.HTM (дата обращения: 14 мая 2026 г.).
- [19] Компьютер маленького человечка. — URL: <https://habr.com/p/257331/> (дата обращения: 14 мая 2026 г.).
- [20] Многопроцессорный учебный компьютер "E14". — URL: <https://emc.orgfree.com/e14/> (дата обращения: 14 мая 2026 г.).
- [21] Модель учебной ЭВМ. — URL: <https://emc.orgfree.com/model/> (дата обращения: 14 мая 2026 г.).
- [22] ОТЧЕТ ПО РЕЗУЛЬТАТАМ МОНИТОРИНГА (ИССЛЕДОВАНИЯ) РЫНКА ТРУДА В СФЕРЕ МИКРОЭЛЕКТРОНИКИ. — URL: [pdfhttps://spknano.ru/upload/%D0%9E%D1%82%D1%87%D0%B5%D1%82%20%D0%98%D1%81%D0%](https://spknano.ru/upload/%D0%9E%D1%82%D1%87%D0%B5%D1%82%20%D0%98%D1%81%D0%)

BB%D0%B5%D0%B4%D0%BE%D0%B2%D0%B0%D0%BD%D0%B8%D0%B5%20%D1%80%D1%8B%D0%BD%D0%BA%D0%B0%20%D1%82%D1%80%D1%83%D0%B4%D0%B0%20%D0%BD%D0%B0%20%D1%81%D0%B0%D0%B9%D1%82.pdf (дата обращения: 14 мая 2026 г.).

- [23] Операционные системы (рынок России). — URL: [https://www.tadviser.ru/index.php/%D0%A1%D1%82%D0%B0%D1%82%D1%8C%D1%8F:%D0%9E%D0%BF%D0%B5%D1%80%D0%B0%D1%86%D0%B8%D0%BE%D0%BD%D0%BD%D1%8B%D0%B5_%D1%81%D0%B8%D1%81%D1%82%D0%B5%D0%BC%D1%8B_\(%D1%80%D1%8B%D0%BD%D0%BE%D0%BA_%D0%A0%D0%BE%D1%81%D1%81%D0%B8%D0%B8\)](https://www.tadviser.ru/index.php/%D0%A1%D1%82%D0%B0%D1%82%D1%8C%D1%8F:%D0%9E%D0%BF%D0%B5%D1%80%D0%B0%D1%86%D0%B8%D0%BE%D0%BD%D0%BD%D1%8B%D0%B5_%D1%81%D0%B8%D1%81%D1%82%D0%B5%D0%BC%D1%8B_(%D1%80%D1%8B%D0%BD%D0%BE%D0%BA_%D0%A0%D0%BE%D1%81%D1%81%D0%B8%D0%B8)) (дата обращения: 14 мая 2026 г.).